

Mock 2 MCQ Question

Medium + Hard in Section 1 • Easy in Section 2 • Correct answers bolded

SECTION 1 — MEDIUM + HARD

Q21. [Medium] Which query adds a new column 'email' to the 'customers' table?

- A. UPDATE customers ADD email VARCHAR(100);
- B. ALTER TABLE customers ADD COLUMN email VARCHAR(100);**
- C. INSERT INTO customers ADD email VARCHAR(100);
- D. ALTER customers ADD email VARCHAR(100);

Q22. [Medium] A large e-commerce platform collects user activity logs, including clicks, search queries, and purchase history. The company wants to analyze this data to generate insights about user behavior across different time periods. Which part of the data architecture is most suitable for this type of analysis?

- A. OLTP Database
- B. Data Warehouse**
- C. NoSQL Document Store
- D. Data Lake

Q23. [Medium] A retail chain is tracking real-time inventory updates, customer orders, and payments. The company requires frequent updates and quick responses on individual transactions. Which system would best support these operations?

- A. Operational System
- B. OLTP System**
- C. NoSQL Database
- D. OLAP System

Q24. [Medium] A manager needs to run a query to retrieve sales details but wants to avoid repeating the same subquery logic twice. The manager decides to use a CTE to simplify the query. Which pattern uses a CTE correctly?

- A. SELECT * FROM customer_totals JOIN orders o ON customer_totals.customer_id = o.customer_id;
- B. WITH customer_totals AS (SELECT customer_id, SUM(amount) FROM sales GROUP BY customer_id);
- C. WITH customer_totals AS (SELECT customer_id, SUM(amount) AS total_amount FROM sales GROUP BY customer_id) SELECT c.customer_id, total_amount, o.region FROM customer_totals c JOIN orders o ON c.customer_id = o.customer_id;**
- D. WITH customer_totals AS (SELECT customer_id, SUM(amount) AS total_amount FROM sales) SELECT * FROM customer_totals;

Q25. [Medium] What will be the output of the following Python code?

```
x = 7
y = 10
if x > y:
    print('Larger')
elif x == y:
    print('Equal')
else:
    print('Smaller')
```

- A. Larger
- B. Equal
- C. Error
- D. Smaller**

Q26. [Medium] A report needs to display the top 3 highest-selling products and their total revenue. You decide to use a CTE to extract the total revenue per product and then sort it to get the top 3. Which query structure would you use?

- A. `SELECT product_id, SUM(revenue) AS total_revenue FROM sales GROUP BY product_id ORDER BY revenue DESC LIMIT 3;`
- B. `SELECT * FROM sales GROUP BY product_id ORDER BY revenue DESC LIMIT 3;`
- C. `SELECT * FROM sales WHERE product_id IN (SELECT product_id FROM sales GROUP BY revenue);`
- D. `WITH top_products AS (SELECT product_id, SUM(revenue) AS total_revenue FROM sales GROUP BY product_id) SELECT * FROM top_products ORDER BY total_revenue DESC LIMIT 3;`**

Q27. [Hard] You want to find all products that are priced higher than the average price of products in the 'Electronics' category. Which query uses a subquery?

- A. `SELECT * FROM products WHERE price = (SELECT AVG(price) FROM products WHERE category = 'Electronics');`
- B. `SELECT * FROM products WHERE price > ALL (SELECT AVG(price) FROM products WHERE category = 'Electronics');`
- C. `SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products WHERE category = 'Electronics');`**
- D. `SELECT * FROM products WHERE price >= (SELECT AVG(price) FROM products WHERE category = 'Electronics');`

Q28. [Hard] You need to display a report of all customers and their orders. Customers who haven't placed any orders should also be shown with NULL values in the orders column. Which query structure should you use?

- A. `SELECT customers.customer_id, orders.order_id FROM customers INNER JOIN orders ON customers.customer_id = orders.customer_id;`
- B. `SELECT customers.customer_id, orders.order_id FROM customers LEFT JOIN orders ON customers.customer_id = orders.customer_id;`**
- C. `SELECT customers.customer_id, orders.order_id FROM customers FULL JOIN orders ON customers.customer_id = orders.customer_id;`
- D. `SELECT customers.customer_id, orders.order_id FROM customers RIGHT JOIN orders ON customers.customer_id = orders.customer_id;`

Q29. [Hard] You have a dataset `df` with several columns, including one titled `sales`. You want to find the average of the `sales` column only for rows where the product category is 'Electronics'. Which operation in Pandas would you use?

- A. `df['sales'].groupby('category').mean()`
- B. `df['sales'].mean()`
- C. `df[df['category'] == 'Electronics']['sales'].mean()`**
- D. `df.groupby('category')['sales'].mean()`

Q30. [Hard] You need to calculate the total sales for each product. The sales data includes a `product_id` and `amount` in the `sales` table. You want to join it with the `products` table to display the product name along with its total sales. Which query correctly achieves this?

- A. `SELECT products.product_name, SUM(sales.amount) FROM sales LEFT JOIN products ON sales.product_id = products.product_id GROUP BY products.product_name;`
- B. `SELECT products.product_name, sales.amount FROM sales JOIN products ON sales.product_id = products.product_id;`
- C. `SELECT products.product_name, COUNT(sales.amount) FROM sales LEFT JOIN products ON sales.product_id = products.product_id GROUP BY products.product_name;`
- D. `SELECT products.product_name, SUM(sales.amount) FROM sales JOIN products ON sales.product_id = products.product_id GROUP BY products.product_name;`**

SECTION 2 — EASY

Q1. [Easy] What is the result of `len([10, 20, 30, 40])`?

- A. 3
- B. 4**
- C. Error
- D. [10, 20, 30, 40]

Q2. [Easy] Which statement is true about nested if conditions in Python?

- A. A nested if runs only if the outer condition is true**
- B. Nested if statements must end with an else
- C. Python does not allow nested if statements
- D. Nested if statements are faster than normal if

Q3. [Easy] Which SQL keyword is used to count the number of rows in a result set?

- A. COUNT(ALL)
- B. SUM
- C. COUNT(*)**
- D. TOTAL

Q4. [Easy] Which code snippet correctly plots a simple line chart using Matplotlib?

- A. `plt.lineplot([1, 2, 3]); plt.show()`
- B. `plt.plot([1, 2, 3]); plt.show()`**
- C. `plt.drawline([1, 2, 3]); plt.show()`
- D. `plt.graph([1, 2, 3]); plt.show()`

Q5. [Easy] Which query returns all employees with a salary greater than 50000?

- A. `SELECT * FROM employees WHERE salary = 50000;`
- B. `SELECT salary > 50000 FROM employees;`
- C. `SELECT * FROM employees HAVING salary > 50000;`
- D. `SELECT * FROM employees WHERE salary > 50000;`**

Q6. [Easy] You want to simplify a long query on a sales table by first filtering high-value rows and then selecting from that filtered set. You decide to use a CTE named `high_sales` that returns rows where `amount > 1000`. After defining the CTE, you want to select all columns from it. Which SQL snippet uses a CTE correctly?

- A. `WITH high_sales (SELECT * FROM sales WHERE amount > 1000) SELECT * FROM high_sales;`
- B. `SELECT * FROM sales WITH high_sales AS (amount > 1000);`
- C. `SELECT * FROM high_sales; WITH high_sales AS (SELECT * FROM sales WHERE amount > 1000);`
- D. `WITH high_sales AS (SELECT * FROM sales WHERE amount > 1000) SELECT * FROM high_sales;`**

Q7. [Easy] What is the output of the following code?

```
nums = [5, 10, 15, 20, 25]
print(nums[-4:-1])
```

- A. [10, 15, 20]**
- B. [10, 20, 25]
- C. [5, 10, 15]
- D. [15, 20, 25]

Q8. [Easy] Which Python keyword is used after the try block to handle an exception?

- A. warn
- B. except**
- C. error
- D. catch

Q9. [Easy] What does unit testing focus on in software development?

- A. Testing user interfaces
- B. Testing individual components or functions**
- C. Testing the entire application
- D. Testing system performance

Q10. [Easy] Which statement is true when creating a table in SQL?

- A. A table can be created without defining any columns
- B. A table's primary key can only be defined after the table is created
- C. You can create a table with only a column name but no data type
- D. A table requires at least one column to be defined**

Q11. [Easy] What is the output of the following code?

```
x = 12
if x > 15:
    print('High')
elif x > 10:
    print('Medium')
else:
    print('Low')
```

- A. Low
- B. Error
- C. Medium**
- D. High

Q12. [Easy] What will be printed by the following code?

```
x = 10
if x > 5:
    print('Greater')
elif x == 10:
    print('Equal')
else:
    print('Smaller')
```

- A. Smaller
- B. Greater
- C. Equal
- D. Error**

Q13. [Easy] To create a histogram in Matplotlib, which function should be used?

- A. plt.plot()
- B. plt.line()
- C. plt.bar()
- D. plt.hist()**

Q14. [Easy] How do you handle multiple types of exceptions in Python?

- A. Using the error keyword
- B. Using multiple finally blocks
- C. Using multiple try blocks
- D. Using multiple except blocks**

Q15. [Easy] Which function is used to set the axes labels in a Matplotlib chart?

- A. plt.set_labels()
- B. plt.axis_labels()
- C. plt.label_axes()
- D. plt.xlabel() and plt.ylabel()**

Q16. [Easy] Which of the following code snippets correctly creates a line graph in Matplotlib?

- A. plt.display([1, 2, 3])
- B. plt.plot([1, 2, 3])**
- C. plt.show_line([1, 2, 3])
- D. plt.graph([1, 2, 3])

Q17. [Easy] What is the output of the following Python code?

try:

result = 5 / 'a'

except TypeError:

print('Cannot perform operation with string')

- A. Cannot perform operation with string**
- B. TypeError occurs and is handled
- C. Error
- D. Nothing

Q18. [Easy] Which of the following is the correct way to handle multiple exceptions in Python?

- A. Using only the try block
- B. Using one catch block
- C. Using different except blocks for different errors**
- D. Using a single except block for all errors

Q19. [Easy] Which SQL query returns the average salary of employees in department 7?

- A. GROUP BY dept_id
- B. SELECT AVG(salary) FROM employees GROUP BY dept_id;
- C. SELECT salary FROM employees WHERE dept_id = 7;
- D. SELECT AVG(salary) FROM employees WHERE dept_id = 7;**

Q20. [Easy] Which query returns all employees whose salary is greater than the average salary?

- A. SELECT * FROM employees WHERE salary = ALL (SELECT AVG(salary) FROM employees);
- B. SELECT * FROM employees WHERE salary > ALL (SELECT AVG(salary) FROM employees);
- C. SELECT * FROM employees WHERE salary >= (SELECT AVG(salary) FROM employees);
- D. SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);**